

Contents

1	Bash, automatización y procesamiento de texto	1
1.1	De la orden repetida al procedimiento automático	1
1.2	Fundamentos de Bash: órdenes, opciones y variables	2
1.2.1	Anatomía de una orden	2
1.2.2	Variables: asignar y recuperar un valor	2
1.2.3	Sustitución de órdenes, subshells y aritmética	3
1.2.4	Separadores de línea y control de flujo	3
1.3	Caracteres especiales y comillas	3
1.4	Flujos estándar: stdin, stdout, stderr	4
1.5	Redirección: escribir, añadir, leer y separar errores	5
1.6	Tuberías: componer programas sin que se conozcan entre sí	5
1.6.1	El peligro del estado de salida en tuberías: <code>pipefail</code>	5
1.7	De la orden interactiva al script	7
1.7.1	De la idea al pseudocódigo	7
1.7.2	El Shebang y las opciones de seguridad (<code>set -euo pipefail</code>)	7
1.8	Variables, comillas y argumentos posicionales	8
1.9	Aplicación: proteger una operación destructiva	8
1.10	Condicionales	9
1.10.1	<code>[]</code> frente a <code>[[]]</code>	9
1.10.2	Álgebra booleana y operadores de prueba	9
1.10.3	<code>if/elif/else</code>	10
1.11	Bucles	10
1.11.1	<code>for</code> : iterar un número conocido de veces	10
1.11.2	<code>while</code> y lectura línea a línea	11
1.11.3	<code>break</code> y <code>continue</code>	11
1.12	Funciones	11
1.13	Arrays y expansión de llaves	12
1.13.1	Arrays y recorrido seguro de ficheros	12
1.13.2	Expansión de llaves	12
1.14	Operadores nativos de cadena	12
1.15	Hito de consolidación: del programa robusto al texto biológico	13
1.16	El formato FASTA como primer registro multilínea	13
1.17	Buscar con <code>grep</code>	15
1.17.1	Opciones avanzadas y control de contexto	15
1.18	Expresiones regulares	16
1.18.1	Fundamento y sintaxis extendida (ERE)	16
1.18.2	Codicia, grupos de captura y aserciones de contexto (<i>Lookaround</i>)	16
1.18.3	Glob frente a regex; dialectos BRE, ERE y PCRE	17
1.19	Utilidades clásicas de texto: <code>cut</code> , <code>sort</code> , <code>uniq</code> y <code>tr</code>	17
1.20	Transformar con <code>sed</code>	18

1.20.1	La tríada Unix: roles complementarios	18
1.21	Procesar texto estructurado con awk	19
1.21.1	El puente de Shell a awk : conteo de nucleótidos	19
1.22	Lo que viene después: de procesar texto a razonar sobre su coste	20
1.23	Método y errores frecuentes	20
1.24	Síntesis de referencia	20
1.25	Glosario esencial	21
1.26	Fuentes y límites de uso	21
Anexo práctico · Tres programas, un puente a sed y otro a awk		23

Chapter 1

Bash, automatización y procesamiento de texto

1.1 De la orden repetida al procedimiento automático

En el primer capítulo aprendimos los cimientos del entorno Unix: cómo comunicarnos con el sistema operativo a través de una terminal de texto, cómo orientarnos en la jerarquía del sistema de archivos y cómo ejecutar programas con seguridad. Ese conocimiento nos permite resolver con soltura cualquier tarea puntual: inspeccionar un archivo, mover un conjunto de datos o lanzar una herramienta bioinformática.

Sin embargo, la investigación biológica moderna rara vez se enfrenta a un fichero aislado. En proyectos reales de genómica, transcriptómica o metagenómica, los datos de secuenciación masiva se distribuyen en decenas o cientos de muestras de secuenciación (*FASTQ*), con miles de secuencias (*FASTA*) y extensas tablas de anotación genómica (*GFF* o *VCF*). Escribir órdenes una por una a mano sobre veinte ficheros no solo es una tarea tediosa y agotadora; es, ante todo, una fuente garantizada de errores humanos y una pérdida completa de reproducibilidad científica. Una orden tecleada a mano en la terminal se desvanece en cuanto cerramos la sesión; no queda constancia verificable de qué parámetros se usaron ni de cómo se obtuvieron los resultados.

Existe una regla de oro en computación científica: **si tienes que repetir un procedimiento más de dos veces, automatízalo**. Este capítulo sigue el orden del programa oficial de la asignatura (*Módulo 2: Shell Scripting y Procesamiento de Texto*) y está diseñado para transformar al estudiante de un usuario que ejecuta órdenes reactivas a un científico computacional capaz de diseñar procedimientos automáticos, robustos y verificables:

- **Fundamentos de Bash y sintaxis segura:** anatomía de órdenes, variables, caracteres especiales, comillas y aritmética entera.
- **Flujos y composición:** los tres canales estándar de Unix (`stdin`, `stdout`, `stderr`), operadores de redirección y la composición modular mediante tuberías (`|`).
- **De la orden al script:** cómo estructurar la lógica previa en pseudocódigo, empaquetar instrucciones en un script ejecutable, depurar con `bash -n` y blindar su ejecución con `set -euo pipefail`.
- **Estructuras de control y modularidad:** condicionales (`if/elif/else`, `[[ ]]`, `(( ))`), bucles (`for`, `while read`, `break`, `continue`), funciones con ámbito local y operadores nativos de manipulación de cadenas.

- **La tríada Unix de procesamiento de texto:** el formato biológico *FASTA*, búsqueda con **grep** y expresiones regulares extendidas (*ERE*), transformación de secuencias con **sed** y análisis estructurado por columnas con **awk**.

Al finalizar el capítulo, se abrirá formalmente el puente hacia la algoritmia y el análisis del coste computacional, tema central del capítulo TC-CH03.

La ruta **esencial** cubre los fundamentos de órdenes y variables, caracteres especiales, flujos, redirección, tuberías, scripting básico, condicionales, bucles, formato *FASTA*, **grep** y **sed**; 190–230 minutos. La ruta de **estudio** profundiza en funciones, expansión de llaves, operadores avanzados de cadena, utilidades de texto (**cut**, **sort**, **uniq**, **tr**), **awk** completo con arrays asociativos y la práctica integradora; 140–170 minutos adicionales.

1.2 Fundamentos de Bash: órdenes, opciones y variables

1.2.1 Anatomía de una orden

Toda orden en la Shell de Bash se descompone en tres componentes fundamentales: **nombre_de_orden** [**opciones**] [**argumentos**]. El nombre indica el ejecutable que se desea lanzar; las opciones modifican el comportamiento del programa; y los argumentos representan los datos o rutas sobre los que actuará.

Las opciones suelen presentarse en dos formatos: **forma corta** con un único guion (como **-l** o **-h**) y **forma larga** con dos guiones (como **-all** o **-human-readable**). Cuando varias opciones cortas no requieren argumento propio, pueden combinarse tras un único guion (**-lh** equivale exactamente a **-l -h**).

Una orden de Bash se descompone en nombre de la orden, opciones (forma corta con un guion o larga con dos guiones) y argumentos; el orden relativo de las opciones entre sí normalmente no importa, pero su posición respecto a los argumentos sí puede importar según la orden.

Varias opciones cortas sin argumento propio pueden combinarse tras un único guion: **wc -l** y **wc -lw** piden el mismo resultado a **wc**. Se reproduce con `verification/chapter_results.sh combined_short_options`.

1.2.2 Variables: asignar y recuperar un valor

Una variable es un contenedor con nombre que almacena información en la memoria de la Shell durante la sesión. En Bash, la asignación de una variable se escribe con la sintaxis **NOMBRE=valor**, con una condición indispensable: **no debe haber ningún espacio alrededor del signo igual**. Si se introduce un espacio (**NOMBRE = valor**), Bash interpretará que **NOMBRE** es un comando y que **=** y **valor** son sus argumentos, resultando en un fallo inmediato.

Para recuperar el valor almacenado en una variable se antepone el símbolo **\$**: **\$NOMBRE**. Cuando el nombre de la variable está adyacente a otros caracteres de texto que podrían confundir al analizador sintáctico, es obligatorio y una excelente práctica de ingeniería delimitar el nombre entre llaves: **\${NOMBRE}**.

NOMBRE=valor asigna sin error; **NOMBRE = valor** (con espacios) no asigna la variable y falla o hace otra cosa. Se reproduce con `verification/chapter_results.sh variable_assignment_spacing`.

\${NOMBRE}_sufijo recupera **NOMBRE** y añade el sufijo literal; **\$NOMBRE_sufijo** intenta recuperar una variable distinta que normalmente no existe y se expande a vacío. Se reproduce con `verification/chapter_results.sh brace_disambiguation`.

1.2.3 Sustitución de órdenes, subshells y aritmética

A menudo necesitamos capturar el resultado emitido por una orden para guardarlo en una variable o pasarlo como argumento a otro programa. La sintaxis moderna y anidable de **sustitución de órdenes** es `$(orden)`. Bash ejecuta `orden`, captura su salida estándar (`stdout`), elimina el salto de línea final y sustituye la expresión por ese contenido.

Por otro lado, los paréntesis simples `( ... )` ejecutan el bloque de órdenes en un entorno secundario aislado o **subshell**. Cualquier cambio en variables o en el directorio de trabajo dentro del subshell se descarta al finalizar, sin alterar la sesión principal.

Para realizar cálculos matemáticos enteros, Bash proporciona la **expansión aritmética** mediante la sintaxis `$(( expresión ))`. Es fundamental recordar que la aritmética nativa de Bash es estrictamente **entera**: la división trunca la parte decimal hacia cero, por lo que `$(( 7 / 2 ))` da como resultado 3 y no 3.5.

`$(orden)` se sustituye por el `stdout` de `orden` sin el salto de línea final; el resultado puede asignarse a una variable o usarse directamente como argumento de otra orden. Se reproduce con `verification/chapter_results.sh command_substitution`.

`$(( 7 / 2 ))` vale 3, no 3.5: la aritmética de `$(( ))` es entera y trunca hacia cero, no redondea. Se reproduce con `verification/chapter_results.sh arithmetic_expansion`.

1.2.4 Separadores de línea y control de flujo

Además del salto de línea convencional, Bash reconoce operadores especiales para encadenar múltiples órdenes en una única línea con semántica de control:

- `;` (punto y coma): ejecuta la primera orden y, cuando termina, ejecuta la siguiente de forma incondicional.
- `&&` (Y lógico): ejecuta la segunda orden **únicamente si la primera terminó con éxito** (código de salida 0).
- `||` (O lógico): ejecuta la segunda orden **únicamente si la primera falló** (código de salida distinto de 0).
- `&` (ampersand simple): lanza la orden en segundo plano (*background*) y devuelve el control a la Shell de inmediato.

`cmdA && cmdB` ejecuta `cmdB` solo si `cmdA` termina con estado 0; `cmdA || cmdB` ejecuta `cmdB` solo si `cmdA` termina con estado distinto de cero; `;` encadena sin mirar el resultado; `&` lanza en segundo plano. Se reproduce con `verification/chapter_results.sh line_separators`.

1.3 Caracteres especiales y comillas

En la Shell de Bash, ciertos caracteres tienen un significado especial y se denominan **metacaracteres**. La Shell interpreta estos caracteres antes de entregar los argumentos al programa ejecutable. Dominar el uso de las comillas es la habilidad más importante para escribir código seguro y predecible.

Carácter	Significado en Bash	Prevención
\$	Expansión de variables y sustituciones	'\$VAR' o \ \$VAR
* ? []	Coincidencia de nombres de fichero (<i>globbing</i>)	Entrecomillar o \
~	Expansión de directorio personal (<i>home</i>)	Entrecomillar
#	Inicio de comentario	Entrecomillar
; &	Separadores de órdenes y tuberías	Entrecomillar

Carácter	Significado en Bash	Prevención
espacio	Separador de palabras y argumentos	Comillas dobles "..."
\	Carácter de escape: quita significado al siguiente	\\
'	Comillas simples: suprime toda expansión	Cerrar comilla
"	Comillas dobles: permite \$ y \$(...), pero bloquea división	Cerrar comilla

Los metacaracteres de esta tabla son el conjunto mínimo necesario para leer y escribir con seguridad cualquier script de este capítulo.

La regla general de las comillas en Bash es sencilla pero crucial:

1. **Comillas simples ('...')**: proporcionan literalidad absoluta. Nada dentro de comillas simples se expande: ni variables ('\$VAR'), ni sustituciones ('\$\$(pwd)'), ni comodines ('*.fasta'). Es el escudo más fuerte para preservar texto intacto.
2. **Barra invertida (\)**: escapa un único carácter inmediatamente posterior, anulando su significado especial.
3. **Comillas dobles ("...")**: permiten la expansión de variables ("\$VAR") y sustitución de órdenes ("\$(cmd)"), pero **bloquean la división en palabras (*word splitting*) y la expansión de rutas (*globbing*)**.

Dentro de comillas simples ninguna expansión ocurre; el texto se pasa literalmente carácter a carácter, incluido el signo dólar o el asterisco. Se reproduce con `verification/chapter_results.sh single_quote_literal`.

Una barra invertida antes de un único carácter le quita su significado especial para esa aparición; `\$VAR` imprime literalmente el signo dólar seguido del nombre. Se reproduce con `verification/chapter_results.sh backslash_escape`.

Dentro de comillas dobles sí se expanden `$variable` y `$(orden)`, pero el resultado no se somete a división de palabras ni a expansión de *glob*. Se reproduce con `verification/chapter_results.sh double_quote_expands_blocks_split`.

1.4 Flujos estándar: `stdin`, `stdout`, `stderr`

En la arquitectura de Unix, todo proceso en ejecución nace con tres canales de comunicación unidireccionales abiertos de forma predeterminada, identificados por un número entero o **descriptor de archivo**:

1. **Entrada estándar (`stdin`, descriptor 0)**: canal por el que el programa lee sus datos de entrada. Por defecto, está conectado al teclado.
2. **Salida estándar (`stdout`, descriptor 1)**: canal por el que el programa emite sus resultados normales de procesamiento. Por defecto, está conectado a la pantalla del terminal.
3. **Error estándar (`stderr`, descriptor 2)**: canal por el que el programa emite avisos, diagnósticos y mensajes de error. Aunque por defecto también se muestra en la pantalla, es un canal completamente independiente de `stdout`.

Esta separación conceptual es un principio maestro de ingeniería: permite guardar o canalizar los datos biológicos válidos emitidos por `stdout` sin que queden contaminados por advertencias o diagnósticos de depuración emitidos por `stderr`.

stdin, stdout y stderr son tres flujos independientes de un proceso; separarlos permite conservar un resultado mientras los avisos se descartan o se inspeccionan por separado. Se reproduce con `verification/chapter_results.sh separate_stderr`.

1.5 Redirección: escribir, añadir, leer y separar errores

Los operadores de redirección permiten alterar el destino o el origen de los flujos estándar, conectándolos a archivos en disco en lugar de la terminal.

Operador	Significado y comportamiento	Ejemplo
>	Redirige stdout a fichero, truncando previo	<code>orden > salida.txt</code>
»	Redirige stdout a fichero, añadiendo al final	<code>orden » salida.txt</code>
<	Redirige fichero hacia stdin	<code>orden < entrada.txt</code>
2>	Redirige solo stderr a fichero	<code>orden 2> errores.log</code>
2»	Redirige stderr añadiendo al final	<code>orden 2» errores.log</code>
&>	Redirige ambos (stdout y stderr)	<code>orden &> todo.log</code>

>, », <, 2>/2» y &> son los operadores de redirección documentados en el manual de Bash para este capítulo.

Advertencia sobre la truncación inmediata. El operador > trunca y vacía el archivo de destino en el mismo instante en que la Shell interpreta la orden, **antes** de que el comando empiece a ejecutarse. Si un script intenta leer y escribir sobre el mismo archivo a la vez (`sort muestra.txt > muestra.txt`), el archivo quedará completamente vacío antes de que `sort` pueda leer una sola línea, perdiéndose los datos de forma irreversible.

`orden > fichero` trunca `fichero` antes de escribir, incluso si la orden falla y no produce salida; el contenido previo se pierde. Se reproduce con `verification/chapter_results.sh truncate_on_redirect`.

2> redirige únicamente stderr; una orden ejecutada con salida y error redirigidos por separado escribe su resultado y sus diagnósticos en ficheros distintos. Se reproduce con `verification/chapter_results.sh separate_stderr`.

» añade al final del destino sin truncar el contenido previo, a diferencia de >; dos escrituras sucesivas conservan ambos bloques de datos. Se reproduce con `verification/chapter_results.sh append_does_not_truncate`.

1.6 Tuberías: componer programas sin que se conozcan entre sí

El operador de tubería (`|`, *pipe*) es la encarnación más elegante de la filosofía Unix: conecta directamente la salida estándar (**stdout**) de un programa con la entrada estándar (**stdin**) del siguiente.

En `a | b`, el stdout de `a` se convierte en el stdin de `b`; ni `a` ni `b` necesitan saber que su entrada o salida proviene de una tubería.

1.6.1 El peligro del estado de salida en tuberías: pipefail

Por defecto, en una tubería compuesta (`cmd1 | cmd2 | cmd3`), la Shell asigna a la variable especial `$?` el código de salida del **último comando ejecutado** (`cmd3`). Si el primer comando (`cmd1`) falla catastróficamente (por ejemplo, porque el archivo de datos no existe), pero `cmd3` termina correctamente recibiendo una entrada vacía, la Shell reportará que la tubería tuvo éxito (`$? = 0`).

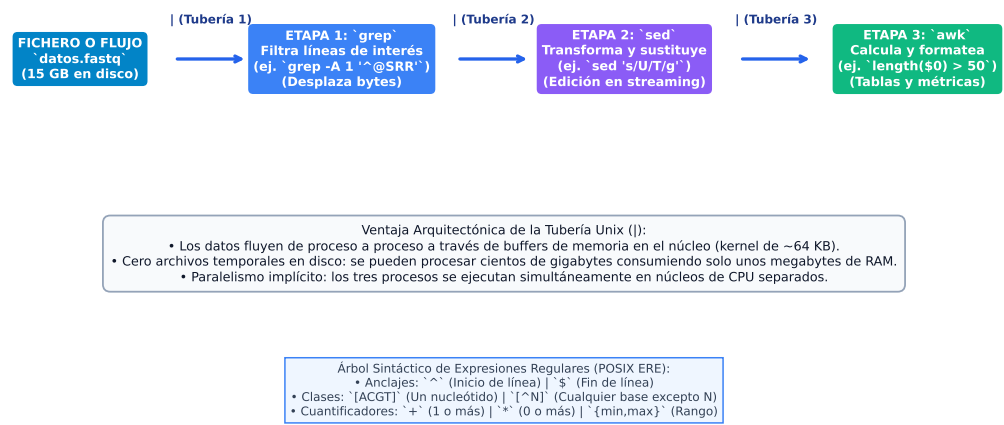


Figure 1.1: Flujograma del modelo de tuberías Unix (1) conectando `grep`, `sed` y `awk` en memoria, junto con el árbol sintáctico de expresiones regulares POSIX ERE. Figura original adaptada de las presentaciones docentes.

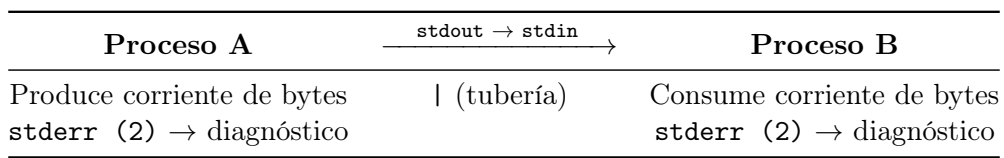


Figure 1.2: Esquema de una tubería Unix. Los programas no se conocen entre sí; solo el flujo de texto que cruza el pipe los acopla.

En bioinformática, este comportamiento por defecto puede arruinar un análisis completo sin dejar rastro de error. Para solucionar esto, activamos la opción `set -o pipefail`. Con `pipefail`, el estado de salida de la tubería será el del comando que haya fallado más a la derecha, garantizando que ningún fallo inicial pase desapercibido.

Sin `set -o pipefail`, el estado de salida de una tubería de varias etapas refleja solo el del último comando; el fallo del primero pasa inadvertido. Se reproduce con `verification/chapter_results.sh pipeline_exit_status`.

1.7 De la orden interactiva al script

1.7.1 De la idea al pseudocódigo

Escribir un script no consiste en teclear órdenes apresuradas en un archivo. El desarrollo profesional comienza siempre diseñando el procedimiento en **pseudocódigo**: una descripción lógica y estructurada en lenguaje natural, libre de particularidades de sintaxis, que descompone el problema biológico en pasos elementales:

Listing 1.1: Ejemplo de pseudocódigo previo al script.

```
Para cada archivo que termine en .fasta dentro de datos/corpus/:
  Comprobar si el archivo contiene datos regulares
  Contar el numero de registros (lineas que empiezan por >)
  Si tiene mas de 100 registros:
    Anotar el archivo en la lista de alta densidad
  Si no:
    Anotar el archivo en la lista estandar
Fin del bucle
```

Escribir primero el procedimiento en pseudocódigo —lenguaje claro, sin sintaxis exacta— antes de traducirlo a Bash reduce errores de diseño y clarifica el flujo de datos.

1.7.2 El Shebang y las opciones de seguridad (`set -euo pipefail`)

Un script de Bash es un archivo de texto con permisos de ejecución (`chmod +x script.sh`). La primera línea debe contener el **shebang**: `#!/usr/bin/env bash`, que indica al sistema operativo qué intérprete debe ejecutar el archivo.

Para garantizar la máxima robustez didáctica y prevenir comportamientos silenciosos erróneos, los scripts de este curso adoptan como norma de trabajo el triplete de seguridad:

Listing 1.2: Cabecera estándar de seguridad en scripts de Bash.

```
#!/usr/bin/env bash
set -euo pipefail
```

- `set -e`: termina la ejecución de inmediato si cualquier orden falla (código de salida distinto de 0).
- `set -u`: termina con error si se intenta leer una variable no definida, evitando que una errata tipográfica se convierta silenciosamente en una cadena vacía.
- `set -o pipefail`: propaga los fallos ocurridos en cualquier eslabón de una tubería.

Límites y tratamiento explícito de excepciones. Esta cabecera no demuestra que el script sea matemáticamente correcto ni sustituye la validación explícita de entradas y precondiciones. Cuando un comando puede devolver código no cero como comportamiento esperado (por ejemplo, cuando `grep` no encuentra coincidencias en una búsqueda condicional), el script debe gestionar ese estado explícitamente mediante una estructura condicional (`if`) o una alternativa controlada (`|| true`), en vez de delegar pasivamente la decisión en `set -e`.

Antes de ejecutar un script, podemos verificar si contiene errores de sintaxis sin lanzarlo realmente utilizando la opción de prueba en seco `bash -n script.sh`. Para seguir la traza completa de ejecución paso a paso, usamos `bash -x script.sh`.

`bash -n script.sh` comprueba la sintaxis sin ejecutar el script; detecta errores sintácticos, pero no de lógica ni de datos. Se reproduce con `verification/chapter_results.sh syntax_check`.

`set -u` hace que el script termine con error al hacer referencia a una variable no definida, en vez de sustituirla silenciosamente por una cadena vacía. Se reproduce con `verification/chapter_results.sh set_u_catches_typo`.

`set -o pipefail` hace que el estado de salida de una tubería sea distinto de cero si cualquiera de sus etapas falla. Se reproduce con `verification/chapter_results.sh pipeline_exit_status`.

1.8 Variables, comillas y argumentos posicionales

Para que un script sea verdaderamente reutilizable, debe aceptar argumentos por línea de comandos. Dentro del script, estos argumentos se reciben en variables especiales denominadas **parámetros posicionales**:

- `$0`: nombre o ruta con la que se invocó el propio script.
- `$1`, `$2`, ..., `$9`: primer, segundo y sucesivos argumentos posicionales.
- `$#`: número total de argumentos recibidos.
- `"$@"`: expande todos los argumentos posicionales manteniendo cada uno como una palabra separada e individual, incluso si contienen espacios internos. Es siempre la forma preferida frente a `$*`.

Dentro de un script, `$0` es el nombre del propio script, `$1` es el primer argumento, `$#` es el número total de argumentos y `"$@"` expande todos los argumentos preservando sus límites. Se reproduce con `verification/chapter_results.sh positional_args`.

Una variable sin comillas que representa una ruta se somete a división de palabras y expansión de *glob*; si contiene espacios o comodines, la orden recibe más o menos argumentos de los previstos. Se reproduce con `verification/chapter_results.sh quoting_matters`.

1.9 Aplicación: proteger una operación destructiva

Una trampa clásica en scripting ocurre al combinar variables con operaciones de borrado (`rm -rf`). Consideremos el siguiente caso:

Listing 1.3: Peligro de borrado por variable vacía y comodín fuera de comillas (NO EJECUTAR).

```
DIRECTORIO_TEMP=""
# ERROR PELIGROSO:
# rm -rf "$DIRECTORIO_TEMP"/* <- Si la variable esta vacia, se expande a rm -rf /*
```

Aunque tengamos activado `set -u`, esta opción solo detecta variables **no declaradas**. Una variable declarada pero asignada a la cadena vacía (`VAR=""`) pasa `set -u` sin emitir ningún error. Al concatenarse con el comodín fuera de las comillas (`"$DIRECTORIO_TEMP"/*`), la expresión se expande a `/*` (la raíz del sistema de archivos), con consecuencias catastróficas. Si el asterisco estuviera dentro de las comillas (`"$DIR/*"`), no expandiría comodines sino que buscaría un nombre literal con asterisco.

Para evitar desastres, seguimos el protocolo preventivo de comprobación:

1. Validar con `[[ -n "$VAR" ]]` que la variable contenga una cadena no vacía.
2. Validar con `[[ -d "$VAR" ]]` que el directorio exista en disco.
3. Realizar siempre una inspección inocua con `printf` antes de cualquier acción: `printf '<%s>\n' "$DIRECTORIO_TEMP"/*`

`set -u` detecta una variable *no definida*, pero no detecta una variable definida y vacía (`VAR=""`); `rm -rf "$VAR/*"` pasa sin error y resulta destructivo sobre un ámbito indeseado. Se reproduce con `verification/chapter_results.sh empty_var_passes_set_u`.

El protocolo de TC-CH01 se completa aquí con un paso adicional específico de scripts parametrizados: mostrar la expansión exacta con un comando inofensivo como `printf` antes de ejecutar cualquier acción destructiva. Se reproduce con `verification/chapter_results.sh destructive_dry_run`.

1.10 Condicionales

1.10.1 `[ ]` frente a `[[ ]]`

Para evaluar condiciones en Bash disponemos del comando clásico `[ ]` (o `test`) y de la construcción moderna extendida `[[ ]]`. En scripts modernos de Bash, **se debe utilizar siempre** `[[ ]]`, ya que evita errores de división de palabras y permite operadores lógicos avanzados como `&&`, `||` y coincidencia de patrones con expresiones regulares (`=`).

Aspecto	<code>[]</code> (clásico / POSIX)	<code>[[]]</code> (Bash moderno)
Comillas en variables	Obligatorias siempre (<code>"\$var"</code>)	Muy recomendadas por estilo
División de palabras	Sí, falla si la variable contiene espacios	No se produce dentro de <code>[[]]</code>
Operadores lógicos	<code>-a</code> (AND), <code>-o</code> (OR)	<code>&&</code> (AND), <code> </code> (OR)
Coincidencia de patrones	No	Sí (<code>== *.txt</code> , <code>= ^[0-9]+\$</code>)

`[ ]` y `[[ ]]` difieren en la necesidad de comillas y en los operadores lógicos que aceptan (`-a/-o` frente a `&&/||`); `[[ ]]` es la forma estándar en Bash.

1.10.2 Álgebra booleana y operadores de prueba

Operador	Significado de la condición
<code>[[-f "\$fichero"]]</code>	Cierto si el fichero existe y es un fichero regular
<code>[[-d "\$dir"]]</code>	Cierto si la ruta existe y es un directorio

Operador	Significado de la condición
<code>[[-s "\$fichero"]]</code>	Cierto si el fichero existe y su tamaño es mayor que cero
<code>[[-r "\$fichero"]]</code>	Cierto si el fichero tiene permiso de lectura
<code>[[-z "\$cadena"]]</code>	Cierto si la cadena de texto está vacía (longitud 0)
<code>[[-n "\$cadena"]]</code>	Cierto si la cadena de texto no está vacía
<code>[["\$a" == "\$b"]]</code>	Comparación de igualdad textual
<code>[["\$a" != "\$b"]]</code>	Comparación de desigualdad textual
<code>((a == b))</code>	Comparación de igualdad numérica
<code>((a > b))</code>	Comparación de desigualdad numérica (mayor que)

Los operadores de cadena, enteros, ficheros y lógicos de esta tabla son el conjunto mínimo necesario para evaluar cualquier condición en los scripts de este capítulo.

`[[ -f "$ruta" ]]` comprueba si `$ruta` es un fichero regular existente; `[[ -z "$var" ]]` comprueba si `var` está vacía; `(( a > b ))` compara `a` y `b` como números, no como texto. Se reproduce con `verification/chapter_results.sh test_operators_string_file_numeric`.

1.10.3 if/elif/else

Listing 1.4: Estructura completa de decisión en Bash.

```
if [[ condicion1 ]]; then
    comando1
elif [[ condicion2 ]]; then
    comando2
else
    comando3
fi
```

`if/elif/else/fi` evalúa condiciones en orden y ejecuta el primer bloque cuya condición sea verdadera; `else` se ejecuta solo si ninguna lo fue.

`(( a > b ))` compara `a` y `b` como números; `[[ "$a" > "$b" ]]` los compara como cadenas de texto y puede dar un resultado distinto ("`9`" > "`10`" es cierto lexicográficamente aunque falso numéricamente). Se reproduce con `verification/chapter_results.sh numeric_vs_lexicographic`.

1.11 Bucles

1.11.1 for: iterar un número conocido de veces

El bucle `for` permite repetir un bloque de instrucciones sobre una colección de elementos o mediante un índice numérico estilo C:

Listing 1.5: Dos formas equivalentes de recorrer 1, 2, 3.

```
for i in 1 2 3; do echo "$i"; done

for ((i=1; i<=3; i+=1)); do echo "$i"; done
```

`for variable in lista; do ... done` y `for ((inicializa; condición; paso)); do ... done` son dos sintaxis equivalentes de `for` en Bash; la segunda es más cómoda cuando se itera con un contador numérico. Se reproduce con `verification/chapter_results.sh for_loop_both_styles`.

1.11.2 while y lectura línea a línea

El bucle `while` repite instrucciones mientras una condición sea cierta. Combinado con el comando `read`, constituye el patrón más eficiente y seguro para procesar archivos de texto gigantescos línea a línea:

Listing 1.6: Patrón seguro de lectura línea a línea en Bash.

```
while IFS= read -r linea; do
    echo "Procesando: $linea"
done < archivo.txt
```

- `IFS=`: suprime el separador de campos por defecto para no recortar espacios en blanco al inicio o final de la línea.
- `-r`: desactiva la interpretación de la barra invertida como escape.

`while IFS= read -r linea; do ... done < fichero` procesa `fichero` línea a línea; `IFS=` evita recortar espacios en los extremos y `-r` evita que la barra invertida se trate como carácter de escape dentro de la línea leída. Se reproduce con `verification/chapter_results.sh while_read_counts_lines`.

1.11.3 break y continue

Para modular el comportamiento de los bucles:

- `break`: interrumpe y finaliza el bucle de inmediato.
- `continue`: interrumpe la iteración actual y salta directamente a la siguiente.

Dentro de un `for`, `break` detiene el bucle antes de procesar los elementos restantes; `continue` omite el resto del cuerpo para el elemento actual pero sigue con el siguiente. Se reproduce con `verification/chapter_results.sh break_and_continue`.

1.12 Funciones

Una función agrupa un conjunto de instrucciones bajo un nombre descriptivo para su reutilización:

Listing 1.7: Definir, invocar y capturar la salida de una función.

```
saludar() {
    local nombre="$1"
    echo "Hola $nombre"
}
saludar "Ana"
resultado="$(saludar "Ana")"
```

Es fundamental distinguir dos conceptos:

1. `return N`: solo devuelve un **código de estado numérico** (de 0 a 255) accesible mediante `$?` . No devuelve texto.
2. `echo` o `printf`: emite datos a la salida estándar, los cuales pueden capturarse en variables mediante `$(funcion)`.

Una función de Bash se define con `nombre() { ... }` y se invoca como una orden; sus argumentos se leen con `$1`, `$2`, etc. Se reproduce con `verification/chapter_results.sh function_define_and_call`.

`return N` termina una función con estado de salida `N` (accesible en `$?`); para devolver datos textuales se imprime con `echo/printf` y se captura con `$(...)`. Se reproduce con `verification/chapter_results.sh function_return_vs_output`.

1.13 Arrays y expansión de llaves

1.13.1 Arrays y recorrido seguro de ficheros

Al recorrer colecciones de archivos con comodines (`*.fasta`), si no hay ningún archivo coincidente, Bash deja por defecto el patrón literal sin expandir (`*.fasta`). Para evitar que el bucle intente procesar un fichero inexistente con ese nombre, activamos `shopt -s nullglob`:

Listing 1.8: Recorrido robusto de ficheros mediante array y `nullglob`.

```
shopt -s nullglob
ficheros=(datos/*.fasta)
if [[ ${#ficheros[@]} -eq 0 ]]; then
    echo "Aviso: no se encontraron archivos FASTA"
fi
```

Con `shopt -s nullglob`, un array construido como `f=(*.fasta)` queda vacío si no hay coincidencias en el directorio, en vez de contener el patrón literal sin expandir. Se reproduce con `verification/chapter_results.sh nullglob_empty_dir`.

1.13.2 Expansión de llaves

La expansión de llaves genera combinaciones y secuencias de texto de forma puramente sintáctica antes de ejecutar la orden:

- Rangos numéricos o de caracteres: `{1..5}` genera `1 2 3 4 5`.
- Listas de alternativas: `muestra_{A,B,C}.txt` genera `muestra_A.txt muestra_B.txt muestra_C.txt`.

`{1..5}` se expande a `1 2 3 4 5` antes de que la Shell ejecute la orden; `{A,B}.txt` genera `A.txt B.txt` sin consultar el sistema de ficheros. Se reproduce con `verification/chapter_results.sh brace_expansion_range_and_list`.

1.14 Operadores nativos de cadena

Bash incluye potentes operadores integrados de manipulación de cadenas que evitan lanzar procesos externos:

- **Extracción de subcadenas:** `${var:pos:long}` extrae `long` caracteres a partir de la posición `pos`.
- **Sustitución de patrones:** `${var/antiguo/nuevo}` sustituye la primera coincidencia; `${var//antiguo/nuevo}` sustituye todas.
- **Eliminación de prefijos:** `${var#prefijo}` elimina el prefijo coincidente más corto; `${var##prefijo}` el más largo.

- **Eliminación de sufijos:** `${var%sufijo}` elimina el sufijo más corto (ideal para quitar extensiones de fichero: `${fichero%.fasta}`).

`${var:0:3}` extrae los tres primeros caracteres de `var`; `${var/e/E}` sustituye la primera coincidencia y `${var//e/E}` sustituye todas. Se reproduce con `verification/chapter_results.sh string_slice_and_substitute`.

`${var#prefijo}` elimina prefijo del inicio de la variable; `${var%sufijo}` elimina sufijo del final (útil para extensiones de fichero). Se reproduce con `verification/chapter_results.sh string_prefix_suffix_removal`.

1.15 Hito de consolidación: del programa robusto al texto biológico

Hito 1 · Dominio de la construcción de programas robustos en Bash. Hasta este punto del capítulo, hemos formalizado las cuatro capacidades fundamentales de la programación en Shell:

1. **Recepción segura de datos:** Manejo de argumentos posicionales con `"$@"` y protección contra separación de palabras y variables vacías.
2. **Estructuras de decisión:** Condicionales robustos con `[[ ... ]]` y evaluación lógica explícita.
3. **Control de flujo y repetición:** Bucles `for` sobre listas y archivos con `nullglob`, y bucles `while read` para flujos línea a línea.
4. **Modularidad:** Definición y llamada a funciones parametrizadas que devuelven resultados limpios.

A partir de la siguiente sección, aplicamos estas herramientas de automatización a nuestro objeto biológico principal: el procesamiento masivo de texto y secuencias genómicas. Para abordar cualquier fichero estructurado, distinguiremos tres operaciones complementarias:

- **Filtrar con grep:** Selecciona líneas enteras que coinciden con un patrón sin alterar su contenido.
- **Transformar con sed:** Modifica, sustituye o recorta el texto línea a línea de forma programática.
- **Procesar con awk:** Descompone cada línea en columnas/campos estructurados, acumulando estado y calculando estadísticas.

Comprobación de integración rápida. Antes de continuar, prediga qué construcción utilizaría en cada caso: (a) ¿Cómo recorrer de forma segura todos los ficheros `*.fasta` de una carpeta sin fallar si no hay ninguno? → Con un array y la opción `shopt -s nullglob`. (b) ¿Cómo seleccionar exclusivamente las líneas de cabecera en un archivo genómico? → Con la expresión regular `^>` en `grep`. (c) ¿Cómo garantizar que un pipeline detecte si falló la primera etapa de una tubería? → Con la directiva `set -o pipefail`.

1.16 El formato FASTA como primer registro multilínea

El formato **FASTA** es el estándar universal en biología computacional para almacenar secuencias de nucleótidos o aminoácidos. Su estructura es estrictamente multilínea:

1. Una **línea de cabecera** que comienza obligatoriamente con el símbolo mayor que (>), seguido de un identificador y una descripción opcional.
2. Una o más **líneas de secuencia** que contienen las letras del alfabeto biológico (A, C, G, T, N, etc.).

Listing 1.9: Estructura típica de un archivo FASTA con dos registros.

```
>seq1 Secuencia humana de prueba
ATGCGATCGATCGATC
GATCGATCGA
>seq2 Secuencia control con bases indeterminadas
ATGCGTNNNNNNCCGT
```

Diferencia entre contar líneas y contar registros. Un archivo FASTA puede tener sus secuencias repartidas en varias líneas. Por tanto, contar líneas con `wc -l` **no** indica cuántas secuencias hay. Para contar registros biológicos reales debemos contar cuántas líneas comienzan por >, usando `grep -c '^>' fichero.fasta`.

Un registro FASTA consiste en una línea de cabecera que empieza por > seguida de una o más líneas de secuencia hasta la siguiente cabecera o el final del fichero.

`grep -c '^>' fichero.fasta` cuenta el número de **registros** del fichero, no su número de líneas. Se reproduce con `verification/chapter_results.sh fasta_header_count`.

Listing 1.10: Algoritmo formal en pseudocódigo para el procesamiento streaming de archivos FASTA multilínea.

```
ALGORITMO: Parser_FASTA_Multilinea_Streaming
ENTRADA:
- Flujo_Texto: Stream secuencial de líneas de entrada (stdin o archivo)
SALIDA:
- Stream_Registros: Tuplas emitidas (Cabecera, Secuencia_Completa, Longitud,
  Porcentaje_GC)
COMPLEJIDAD: Tiempo O(N) donde N es el total de caracteres, Espacio O(L_max) para el
  buffer

1. INICIALIZACION:
  Cabecera_Actual <- ""
  Buffer_Secuencia <- ""

2. BUCLE STREAMING:
  Para cada Linea en Flujo_Texto:
    Linea <- Eliminar_Espacios_Extremos(Linea)
    Si Linea es vacía entonces:
      Continuar al siguiente ciclo

    Si Linea comienza por '>' entonces:
      # Si teníamos un registro previo acumulado, lo procesamos y emitimos
      Si Cabecera_Actual != "" entonces:
        Longitud <- Longitud_Cadena(Buffer_Secuencia)
        Conteo_GC <- Contar_Ocurrencias(Buffer_Secuencia, {'G', 'C', 'g', 'c'})
        Porcentaje_GC <- (Conteo_GC / Longitud) * 100.0
        EMITIR (Cabecera_Actual, Buffer_Secuencia, Longitud, Porcentaje_GC)

      # Inicializar nuevo registro
      Cabecera_Actual <- Linea
      Buffer_Secuencia <- ""
  Sino:
```



```

# Acumular linea de nucleotidos/aminoacidos en el buffer
Buffer_Secuencia <- Concatenar(Buffer_Secuencia, Linea)

3. TERMINACION (ULTIMO REGISTRO):
Si Cabecera_Actual != "" y Buffer_Secuencia != "" entonces:
  Longitud <- Longitud_Cadena(Buffer_Secuencia)
  Conteo_GC <- Contar_Ocurrencias(Buffer_Secuencia, {'G', 'C', 'g', 'c'})
  Porcentaje_GC <- (Conteo_GC / Longitud) * 100.0
  EMITIR (Cabecera_Actual, Buffer_Secuencia, Longitud, Porcentaje_GC)

```

1.17 Buscar con grep

grep (*Global Regular Expression Print*) es la herramienta reina para buscar y filtrar patrones de texto en Unix.

Líneas coincidentes frente a ocurrencias totales. Si una línea de secuencia contiene tres repeticiones del motivo buscado, **grep -c** devolverá 1 (esa línea contiene al menos una coincidencia). Para contar las ocurrencias individuales exactas debemos extraer cada una con **grep -o** y contar con **wc -l**.

grep -c PATRÓN fichero cuenta **líneas** que contienen al menos una coincidencia, no el número total de coincidencias; una línea con dos coincidencias cuenta como una sola. Se reproduce con `verification/chapter_results.sh grep_count_vs_occurrences`.

-n, -i, -v, -c, -o, -q, -r/-R y -l son las opciones de **grep** documentadas en el manual de GNU Grep para este capítulo. **egrep** es la forma histórica equivalente a **grep -E**.

1.17.1 Opciones avanzadas y control de contexto

Opción	Efecto	Ejemplo
-A n/-B n/-C n	líneas de contexto después/antes/ambos	grep -A2 '^>' f.fasta
-w	exige coincidencia de palabra completa	-w 'AT' no coincide con ATG
-x	exige que la línea entera coincida	grep -x 'ATGC' f.txt
-e PATRÓN (repetible)	añade un patrón más, en relación O	-e A -e T coincide con líneas que tengan A o T
-F	trata el patrón como cadena literal, no regex	un punto deja de ser comodín

La tabla de opciones extendidas de **grep** resume el control de contexto (**-A/-B/-C**), delimitación de palabra (**-w**), línea completa (**-x**), patrones múltiples (**-e**) y búsqueda literal rápida (**-F**).

grep -A n/-B n/-C n añaden a la salida las **n** líneas posteriores, anteriores o ambas alrededor de cada coincidencia. Se reproduce con `verification/chapter_results.sh grep_context`.

grep -w 'AT' no coincide con una línea que contiene ATG, aunque sí coincide con una línea donde AT está rodeado de espacios o al final de línea. Se reproduce con `verification/chapter_results.sh grep_word_boundary`.

grep -x PATRÓN solo cuenta como coincidencia una línea donde el patrón, anclado, cubre la línea completa. Se reproduce con `verification/chapter_results.sh grep_exact_line`.

grep -e PATRON1 -e PATRON2 fichero es equivalente a **grep -E 'PATRON1|PATRON2' fichero**: ambas formas devuelven las líneas que coinciden con al menos uno de los dos patrones. Se reproduce con `verification/chapter_results.sh grep_multiple_patterns`.

`grep -F` trata el patrón como cadena literal, no como expresión regular; un punto busca el carácter punto exacto y no actúa como comodín. Se reproduce con `verification/chapter_results.sh grep_fixed_string`.

1.18 Expresiones regulares

1.18.1 Fundamento y sintaxis extendida (ERE)

Las expresiones regulares (*REGEX*) son un lenguaje formal para describir patrones en cadenas de texto, originadas en los trabajos sobre autómatas finitos del matemático Stephen Kleene (1951) e introducidas en Unix por Ken Thompson en el editor `ed` (1968) y en `grep` (1973).

Las expresiones regulares derivan del álgebra de autómatas finitos formalizada por Kleene (1951) e implementada en Unix por Thompson (1968, 1973).

Construcción	Significado	Ejemplo biológico
<code>^</code>	Ancla de inicio de línea	<code>^></code> (cabeceras FASTA)
<code>\$</code>	Ancla de fin de línea	<code>TAA\$</code> (codón de parada al final)
<code>.</code>	Cualquier carácter excepto salto de línea	<code>A.T</code> (A seguido de cualquier base y T)
<code>[]</code>	Clase de caracteres	<code>[AG]</code> (purinas)
<code>[^]</code>	Clase negada	<code>[^ACGT]</code> (bases no canónicas)
<code>*</code>	Cuantificador: 0 o más veces	<code>N*</code>
<code>+</code>	Cuantificador: 1 o más veces	<code>N+</code> (islas de indeterminación)
<code>?</code>	Cuantificador: 0 o 1 vez (opcional)	<code>AUG?</code>
<code>{n,m}</code>	Entre <i>n</i> y <i>m</i> repeticiones	<code>N{2,}</code> (dos o más bases N)
<code>()</code>	Grupo de captura	<code>(CG){2,}</code> (islas CpG)
<code> </code>	Alternancia (O lógico)	<code>TAA TAG TGA</code> (codones de parada)

Anclas, clases, cuantificadores (incluido `?` para cero o una aparición), grupos y alternancia son las cinco construcciones mínimas de ERE para describir patrones sobre secuencias de texto.

Sobre `data/tc02_caso_regex.fasta`, el patrón `N{2,}` coincide con la línea de secuencia de `caso_dos` y no con la de `caso_uno`. Se reproduce con `verification/chapter_results.sh regex_positive_negative`.

1.18.2 Codicia, grupos de captura y aserciones de contexto (*Lookaround*)

Por defecto, los cuantificadores en REGEX son **voraces** (*greedy*): consumen la mayor cantidad posible de caracteres que satisfagan la condición. En motores compatibles con PCRE o lenguajes modernos, añadir `?` tras un cuantificador (como `.*?`) lo convierte en **perezoso** (*lazy*), consumiendo el mínimo imprescindible.

Los **grupos de captura** con paréntesis (`...`) aíslan subcoincidencias para su reutilización mediante referencias hacia atrás (`\1`, `\2`).

Por último, en bioinformática es muy común buscar un motivo que deba estar precedido o seguido por una secuencia concreta, pero sin que esa secuencia sea consumida en el resultado. Para ello se emplean las construcciones de **ancho cero** (*lookaround*):

- **Positive Lookahead** (`?=patrón`): comprueba que lo siguiente coincida con `patrón`, sin incluirlo.
- **Positive Lookbehind** (`?<=patrón`): comprueba que lo precedente coincida con `patrón`, sin incluirlo.

Los cuantificadores son codiciosos por defecto en la mayoría de los motores de expresiones regulares, consumiendo la subcadena más larga posible.

Un grupo de captura (...) en una expresión regular aísla subcadenas para referencias hacia atrás o sustitución estructurada.

Sin escapar, el punto de una ERE coincide con cualquier carácter; \. coincide solo con el punto literal. Se reproduce con `verification/chapter_results.sh regex_dot_escape`.

[[[:alpha:]]], [[[:digit:]]] y [[[:space:]]] son clases POSIX con nombre equivalentes a sus conjuntos manuales; un circunflejo al inicio de corchetes ([^0-9]) niega la clase. Se reproduce con `verification/chapter_results.sh regex_posix_class_and_negation`.

1.18.3 Glob frente a regex; dialectos BRE, ERE y PCRE

Es crucial no confundir los comodines de la Shell (*globbing*) con las expresiones regulares:

- **Globbing:** lo interpreta la **Shell** para generar nombres de **archivos** antes de lanzar un comando (`*.fasta`).
- **Regex:** lo interpreta el **programa** (`grep`, `sed`, `awk`) para buscar patrones en el **texto**.

Un patrón de *glob* de Bash (`*.fasta`) y una expresión regular no son el mismo lenguaje: el *glob* lo interpreta la Shell sobre nombres de fichero, mientras que la expresión regular la interpreta el programa sobre el contenido de texto.

Dialecto	Dónde se usa	Características
BRE	<code>grep</code> básico, <code>sed</code> por defecto	+, ?, () y { } requieren barra: \+ Sintaxis limpia y moderna sin escapar metacaracteres
ERE	<code>grep -E</code> , <code>sed -E</code> , <code>awk</code>	
PCRE	<code>grep -P</code> , utilidades avanzadas	Añade atajos (\d, \w), lookahead y no codicia

Este capítulo declara y usa siempre ERE; PCRE añade atajos de clase, cuantificadores no codiciosos y aserciones de contexto (*lookaround*); `sed` usa BRE por defecto y requiere `-E` para usar la misma sintaxis.

1.19 Utilidades clásicas de texto: cut, sort, uniq y tr

Unix cuenta con un repertorio de utilidades clásicas indispensables para procesar tablas y listas:

- `cut -d DELIM -f CAMPO`: extrae columnas específicas de un fichero estructurado.
- `sort`: ordena líneas alfabéticamente (o numéricamente con `-n`).
- `uniq`: elimina líneas duplicadas consecutivas y cuenta frecuencias con `-c`. **Regla obligatoria:** `uniq` requiere que la entrada haya sido ordenada previamente con `sort`.
- `tr`: traduce o elimina caracteres individuales (`tr 'a-z' 'A-Z'`).

`cut -d'\t' -f2` extrae el segundo campo delimitado por tabulador; con un delimitador erróneo devuelve la línea completa sin emitir error. Se reproduce con `verification/chapter_results.sh cut_field`.

`uniq` solo elimina líneas duplicadas **adyacentes**; los duplicados no consecutivos requieren ordenar antes con `sort`. Se reproduce con `verification/chapter_results.sh uniq_needs_sort`.

`tr 'a-z' 'A-Z'` traduce carácter a carácter de la primera clase a la segunda; no reconoce palabras ni límites de campo. Se reproduce con `verification/chapter_results.sh tr_case`.

1.20 Transformar con sed

sed (*Stream Editor*) es un editor de flujo línea a línea. No abre una ventana interactiva; lee el texto, aplica un programa de transformación a cada línea y escribe el resultado en la salida estándar, dejando intacto el archivo original (salvo con la opción de edición en el lugar `-i.bak`).

La orden básica de sustitución es `sed 's/patrón/reemplazo/'` (cambia la primera coincidencia por línea) o `sed 's/patrón/reemplazo/g'` (con el modificador `g`, cambia todas). Si el patrón contiene barras (como en rutas de archivos), podemos usar cualquier otro delimitador: `sed 's#/usr/local#/opt#'`.

Podemos restringir las operaciones a líneas específicas mediante **direcciones**:

- **Por número o rango:** `sed '2,5s/A/T/g' fichero.txt`.
- **Última línea (\$):** `sed '$s/A/T/' fichero.txt` aplica la sustitución exclusivamente a la última línea del archivo. Aquí `$` actúa como dirección posicional de **sed**; dentro de una expresión regular (TAA\$) es un ancla de fin de línea, y en la Shell inicia una variable (\$VAR). Las comillas simples impiden que la Shell interprete el símbolo antes de entregarlo a **sed**.
- **Por coincidencia de patrón:** `sed '/^>/s/ /_/g'` (sustituye espacios solo en las cabeceras FASTA).
- **Eliminación:** `sed '/^$/d'` (elimina líneas vacías).
- **Inserción y adición:** `1i\TEXT0` (inserta antes de la línea 1) y `1a\TEXT0` (añade después).

sed es un editor de flujo: lee su entrada línea a línea, aplica el programa de edición y escribe el resultado en stdout sin modificar el fichero de entrada a menos que se use `-i`.

`sed 's/patrón/reemplazo/'` sustituye únicamente la primera coincidencia de cada línea; las siguientes permanecen intactas. Se reproduce con `verification/chapter_results.sh sed_first_match_only`.

`sed 's/patrón/reemplazo/g'` sustituye todas las coincidencias de cada línea gracias al indicador global `g`. Se reproduce con `verification/chapter_results.sh sed_global_flag`.

sed acepta un delimitador distinto de `/` para la sustitución, evitando escapar las barras de una ruta. Se reproduce con `verification/chapter_results.sh sed_custom_delimiter`.

`sed '2s/.../.../'` actúa solo en la línea 2; con un rango de líneas actúa en ese rango; con una dirección por patrón actúa en las líneas que coinciden; sin dirección se aplica a todas. Se reproduce con `verification/chapter_results.sh sed_addressing`.

`sed '/^$/d'` elimina las líneas completamente vacías de la salida; las demás pasan sin cambios. Se reproduce con `verification/chapter_results.sh sed_delete_blank_lines`.

`sed -i` sobrescribe el fichero de entrada sin guardar copia a menos que se especifique un sufijo de seguridad (`sed -i.bak`); usarlo sin sufijo hace irreversible cualquier error en el patrón. Se reproduce con `verification/chapter_results.sh sed_inplace_backup`.

Las órdenes estructurales de **sed** para borrar (`d`), insertar (`i`), añadir (`a`) y cambiar líneas completas (`c`) permiten transformar el flujo de texto de forma programática. Se reproduce con `verification/chapter_results.sh sed_delete_insert_append_change`.

1.20.1 La tríada Unix: roles complementarios

grep decide qué líneas pasan sin cambiarlas (filtra); **sed** transforma el contenido de cada línea que pasa (transforma); **awk** divide cada línea en campos y aplica lógica de programa (procesa). Encadenar los tres cubre la mayoría de las tareas de procesamiento de texto en bioinformática.

1.21 Procesar texto estructurado con awk

awk (creado por Aho, Weinberger y Kernighan) es un lenguaje de programación completo orientado al procesamiento de datos estructurados en filas (**registros**) y columnas (**campos**). Su estructura básica sigue el paradigma: **patrón { acción }**.

Variables especiales fundamentales:

- **\$0**: contenido completo de la línea actual.
- **\$1, \$2, ..., \$NF**: primer, segundo y último campo de la línea actual.
- **NF**: número total de campos en la línea actual (*Number of Fields*).
- **NR**: número de registro o línea global actual (*Number of Records*).
- **FS**: separador de campos (se define con **-F', '** o **-F'\t'**).
- **Bloques BEGIN y END**: **BEGIN { ... }** se ejecuta una única vez antes de leer los datos; **END { ... }** se ejecuta una única vez tras procesar la última línea.

En **awk**, **NR** es el número de registro actual, **NF** es el número de campos y **\$0** es la línea completa; un bloque **END** se ejecuta exactamente una vez tras procesar toda la entrada. Se reproduce con `verification/chapter_results.sh awk_nr_nf`.

awk -F', ' cambia el separador de campos de espacios/tabuladores a coma para procesar ficheros CSV. Se reproduce con `verification/chapter_results.sh awk_custom_field_separator`.

awk '{suma += \$1} END {print suma}' acumula valores aritméticos de una columna a lo largo de las líneas y emite el total final en el bloque **END**. Se reproduce con `verification/chapter_results.sh awk_arithmetic_accumulate`.

El operador **~** comprueba si un campo coincide con una expresión regular (**\$1 ~ /^A/**); **!~** comprueba que no coincida. Se reproduce con `verification/chapter_results.sh awk_match_operator`.

length(\$0) devuelve la longitud de la línea; **substr(\$0, pos, len)** extrae una subcadena a partir de la posición **pos**. Se reproduce con `verification/chapter_results.sh awk_length_substr`.

sub(patrón, reemplazo, \$0) sustituye la primera coincidencia en el registro; **gsub(patrón, reemplazo, \$0)** sustituye todas. Se reproduce con `verification/chapter_results.sh awk_sub_gsub`.

awk -v nombre=valor '...' declara una variable accesible dentro del programa **awk** con el valor asignado desde la Shell. Se reproduce con `verification/chapter_results.sh awk_dash_v`.

Un array asociativo de **awk** (**count[\$clave]++**) acumula frecuencias por clave sin necesidad de declarar previamente qué claves existirán. Se reproduce con `verification/chapter_results.sh awk_assoc_array`.

1.21.1 El puente de Shell a awk: conteo de nucleótidos

Para una secuencia como **AACG**, la tubería `echo "AACG" | awk '{ for(i=1;i<=length($0);i++) c[substr($0,i,1)]++ } END { for(i=1;i<=4;i++) { s=substr("ACGT",i,1); printf "%s:%d", s, c[s]+0 } print "" }'` imprime el conteo de cada una de las cuatro bases (A, C, G, T), incluidas las ausentes con valor cero (A:2 C:1 G:1 T:0). Se reproduce con `verification/chapter_results.sh bridge_counts`.

1.22 Lo que viene después: de procesar texto a razonar sobre su coste

A lo largo de este capítulo hemos dominado la construcción de tuberías, el uso de expresiones regulares y la automatización mediante scripts en Bash y `awk`. Hemos resuelto el problema de procesar miles de registros biológicos de manera reproducible y sin intervención manual.

Sin embargo, a medida que los conjuntos de datos en biomedicina crecen desde unos pocos megabytes hasta cientos de gigabytes, surge una pregunta crítica: ¿cuánto tiempo y memoria consume cada una de estas soluciones? ¿Por qué ordenar con `sort` antes de `uniq` es rápido para diez mil líneas pero lento para cien millones? Responder a estas preguntas requiere pasar de la mera automatización al **pensamiento algorítmico y al análisis de complejidad**, que es el núcleo fundamental que exploraremos en el capítulo **TC-CH03 (Algoritmos y complejidad)**.

Este capítulo y TC-CH01 preparan explícitamente la transición a TC-CH03: una vez que se sabe automatizar y procesar texto, la pregunta abierta es cuánto cuesta cada construcción cuando el tamaño de los datos crece, que es el objeto de estudio de TC-CH03.

1.23 Método y errores frecuentes

Este capítulo recupera, sin volver a enseñarlo, el ciclo predecir/probar en pequeño/inspeccionar/contrastar/registrar introducido en TC-CH01; aquí se aplica a tuberías, patrones y scripts en vez de a órdenes sueltas.

Síntoma	Causa probable	Comprobación y corrección
<code>grep -c</code> da menos de lo esperado	cuenta líneas, no coincidencias	comparar con <code>grep -o ... wc -l</code>
<code>uniq</code> deja duplicados	no eran adyacentes	<code>sort</code> antes de <code>uniq</code>
<code>[AG]+</code> no coincide	se usó BRE en vez de ERE	usar <code>grep -E</code> y declarar el dialecto
un bucle <code>for</code> no encuentra ficheros	no había coincidencias y falta <code>nullglob</code>	activar <code>shopt -s nullglob</code> y comprobar longitud del array
una tubería parece correcta pese a un fallo	solo se observó el estado del último eslabón	<code>set -o pipefail</code> y probar cada etapa por separado
<code>sed</code> no cambia todas las coincidencias	falta el indicador <code>g</code>	añadir <code>g</code> al final de <code>s///</code>
una función no "devuelve" el texto esperado	se confundió <code>return</code> (estado numérico) con capturar salida	usar <code>\$(función)</code> para texto, <code>return</code> solo para estado

Los síntomas de esta tabla corresponden a los errores de composición más frecuentes observados al automatizar procesamiento de texto con las herramientas de este capítulo.

1.24 Síntesis de referencia

Necesidad	Construcción principal	Pregunta de control
parametrizar	<code>NOMBRE=valor, \$NOMBRE, \$(orden)</code>	¿hay espacios alrededor del <code>=</code> ?
citar con seguridad componer	comillas simples/dobles, <code>\</code> <code> </code> , <code>></code> , <code>»</code> , <code>2></code>	¿quiero expansión o texto literal? ¿qué recibe y produce cada etapa?

Necesidad	Construcción principal	Pregunta de control
decidir	<code>[[]]</code> , <code>if/elif/else</code>	¿comparo como texto o como número?
repetir	<code>for</code> , <code>while</code> , <code>break/continue</code>	¿sé de antemano cuántas veces?
reutilizar	funciones, <code>return</code> frente a <code>\$(función)</code>	¿necesito el estado o el texto?
buscar	<code>grep -n -i -v -c -o -E -A -w -e -r -F</code>	¿cuento líneas o coincidencias y con qué dialecto?
describir patrones	anclas, clases, cuantificadores, grupos	¿el caso negativo también falla como espero?
transformar	<code>sed s///[g]</code> , direcciones, <code>d/i/a/c, -i</code>	¿sustituyo la primera coincidencia o todas?
procesar por registros	<code>awk: NR/NF/FS</code> , funciones, <code>~</code>	¿qué separa un registro de un campo aquí?

La tabla de síntesis enlaza cada necesidad operativa del capítulo con su construcción principal y una pregunta de control verificable.

1.25 Glosario esencial

- **Metacaracter:** carácter con significado especial para la Shell antes de ejecutar la orden.
- **stdin/stdout/stderr:** entrada, salida y error estándar.
- **Tubería:** conexión del stdout de un proceso al stdin de otro.
- **Función:** bloque de órdenes con nombre, reutilizable, con argumentos propios.
- **Bucle:** estructura que repite órdenes mientras se cumple una condición o para una lista de elementos.
- **Expansión de llaves:** generación de texto literal a partir de `{...}` o `{n..m}`, sin consultar el sistema de ficheros.
- **Glob:** patrón de Bash para nombres de fichero, interpretado por la Shell.
- **Regex (ERE):** lenguaje de patrones sobre el contenido de un texto, sintaxis extendida usada con `grep -E`.
- **Grupo de captura:** subpatrón entre paréntesis cuyo texto coincidente puede referenciarse después.
- **Cuantificador voraz (greedy):** cuantificador que coincide con la cadena más larga posible.
- **Aserción de ancho cero (lookaround):** comprobación de contexto que no consume caracteres en el resultado.
- **Stream editor (sed):** editor que procesa texto línea a línea sin modificar el origen salvo petición explícita.
- **Registro / Campo (awk):** fila y columna en el modelo de datos estructurados de `awk`.

1.26 Fuentes y límites de uso

Este capítulo se basa en las presentaciones docentes oficiales de la asignatura (PRES-TC-SHELLSCRIPTING y PRES-TC-REGEX, Álvaro Serrano Navarro), en los manuales de referencia de GNU (GNU-BASH-MANUAL,

GNU-GREP-MANUAL, GNU-SED-MANUAL, GNU-GAWK-MANUAL, GNU-COREUTILS-MANUAL), en la especificación del formato FASTA del NCBI (NCBI-FASTA) y en la bibliografía docente de referencia (*Bioinformatics Data Skills*, Buffalo 2015).

Anexo práctico · Tres programas, un puente a sed y otro a awk

Pregunta, producto y separación de materiales

¿Puede convertir una exploración manual de 21 ficheros en tres scripts seguros, explicables y comprobados, más un paso de transformación con **sed** y otro de conteo con **awk** donde Bash puro se vuelve incómodo? Este laboratorio responde que sí sobre el corpus completo del capítulo.

El producto individual contiene tres scripts Bash ejecutables (extremos de todo FASTA, inventario de TXT, rama según líneas), un script que añade un paso con **sed**, un script que añade un paso con **awk**, una tabla de pruebas (**pruebas.tsv**), un **README.md** y una entrega empaquetada y verificada.

El anexo es **own**, con el corpus completo de **data/corpus/** (21 ficheros, CC0-1.0) como entrada de solo lectura. La clave docente (**instructor_key/**) y las implementaciones de referencia (**verification/practice_reference.sh**) no se distribuyen al alumnado.

Mapa de ruta y carga

Bloque	Minutos	Producto
preparar espacio y predecir	10–15	predicción escrita antes de ejecutar
programa 1: extremos FASTA	25–30	script + pruebas.tsv parcial
programa 2: inventario TXT	20–25	script + file_list.txt/summary.txt
programa 3: rama por líneas	25–30	script + casos de frontera probados
paso sed : normalizar cabeceras	15–20	script + conteo contrastado a mano
paso awk : conteo de bases	15–20	script + valor contrastado a mano
empaquetar y verificar	15–20	entrega comprobada

La ruta esencial de este anexo suma 140–175 minutos y cubre los tres programas históricos ampliados con un paso de **sed** y otro de **awk**.

1 · Programa 1: extremos de todos los FASTA

Tarea. Escriba **scripts/extremos.sh** (sin argumentos) que, para cada **.fasta** de **datos/corpus/**, muestre con etiqueta la primera y la última línea de secuencia, usando un array con **nullglob** para manejar sin error el caso sin coincidencias. Antes de ejecutar, prediga cuántos ficheros procesará su script.

Listing 1.11: Comprobación del programa 1.

```
bash -n scripts/extremos.sh
./scripts/extremos.sh | tee resultados/extremos.txt
```

Sobre `datos/corpus/`, un script correcto de extremos procesa exactamente 11 ficheros `.fasta`, ni más ni menos. Se reproduce con `verification/practice_reference.sh extremes`, cuya implementación de referencia cuenta los ficheros procesados con el mismo patrón `nullglob` enseñado en §TC02-SEC-ARRAYS.

2 · Programa 2: inventario de TXT

Tarea. Escriba `scripts/inventario.sh` que cuente los `.txt` de `datos/corpus/`, liste **solo** ficheros regulares (no directorios) en `resultados/file_list.txt`, y guarde las 10 primeras líneas de esa lista en `resultados/summary.txt`. No analice la salida de `ls` para contar: use `find` con `-type f`.

Sobre `datos/corpus/`, un inventario correcto cuenta 6 ficheros `.txt`, escribe 21 nombres en `file_list.txt` y 10 líneas en `summary.txt`. Se reproduce con `verification/practice_reference.sh inventory`.

3 · Programa 3: decidir según el número de líneas

Tarea. Escriba `scripts/resumir.sh RUTA` (un argumento) que muestre el fichero completo si tiene 20 líneas o menos, y que emita un aviso y muestre solo las 10 primeras y 10 últimas líneas si tiene más de 20. Ante una ruta inexistente, debe terminar con un mensaje de error explícito y estado de salida distinto de cero, nunca con una salida vacía silenciosa.

Listing 1.12: Casos de frontera del programa 3.

```
bash -n scripts/resumir.sh
./scripts/resumir.sh datos/corpus/file_4.fasta
./scripts/resumir.sh datos/corpus/file_5.fasta
./scripts/resumir.sh datos/no_existe.txt; echo "estado=$?"
```

En el corpus, `file_4.fasta` tiene 19 líneas y se muestra completo; `file_5.fasta` tiene 21 y activa el aviso con los extremos; una ruta inexistente termina con estado de salida distinto de cero. Se reproduce con `verification/practice_reference.sh branch` sobre ambos ficheros y con `practice_reference.sh branch_missing` sobre una ruta inexistente.

4 · Paso sed: normalizar cabeceras

Tarea. Escriba `scripts/normalizar.sh` que recorra todos los `.fasta` de `datos/corpus/` y, con `sed`, sustituya el primer espacio de cada línea de cabecera (las que empiezan por `>`) por un guion bajo, dejando las líneas de secuencia sin tocar. Escriba el resultado concatenado en `resultados/normalizado.fasta` sin modificar los ficheros originales de `datos/corpus/`.

Listing 1.13: Comprobación del paso sed.

```
bash -n scripts/normalizar.sh
./scripts/normalizar.sh > resultados/normalizado.fasta
grep -c '^>' resultados/normalizado.fasta
grep '^>' resultados/normalizado.fasta | grep -c ' '
```

Sobre `datos/corpus/`, un script correcto de normalización produce 11 cabeceras transformadas y deja cero espacios en las líneas de cabecera del resultado; las líneas de secuencia permanecen idénticas a las originales. Se reproduce con `verification/practice_reference.sh`

`normalize_headers`. Use `sed` con dirección (`/^>/s/ /_/`), no sobre todas las líneas: aplicar la sustitución sin dirección a una línea de secuencia sin espacios no cambiaría nada, pero aplicarla mal a un formato con espacios en la secuencia sí lo estropearía — declare siempre a qué líneas se aplica.

5 · Paso awk: contar bases totales

Tarea. Escriba `scripts/contar_bases.sh` que recorra todos los `.fasta` de `datos/corpus/` y, para cada uno, use `awk` para sumar la longitud de las líneas de secuencia (excluyendo cabeceras), acumulando un total global. Contraste a mano el resultado sobre uno solo de los ficheros antes de confiar en el total.

La suma de longitudes de secuencia (sin contar cabeceras) de los 11 FASTA de `datos/corpus/` es 2162 bases. Se reproduce con `verification/practice_reference.sh awk_total_bases`.

6 · Empaquetar y verificar la entrega

Listing 1.14: Entrega comprobada.

```
cd ..
tar -czf TC02_entrega.tar.gz TC02_entrega
tar -tzf TC02_entrega.tar.gz | head -n 20
sha256sum TC02_entrega.tar.gz
../practice_assets/check_practice_delivery.sh TC02_entrega TC02_entrega.tar.gz
```

`check_practice_delivery.sh` verifica, con Bash exclusivamente, que los cinco scripts existen y pasan `bash -n`, que `datos/` no fue modificado respecto a su manifiesto, y que el `.tar.gz` se extrae de forma íntegra.

Rúbrica de evaluación

La evaluación de este laboratorio pondera la corrección técnica automatizada, el control explícito de casos límite y la fundamentación conceptual sobre 10 puntos (reparto 30/70 según ADR-001):

- **3,0 puntos · Entrega técnica y verificador (DELIVERY_OK):** Entrega íntegra del paquete `.tar.gz`, los cinco scripts requeridos pasan `bash -n` sin errores sintácticos, los datos de entrada permanecen inalterados y `check_practice_delivery.sh` concluye con éxito.
- **2,5 puntos · Robustez, gestión de ramas y control de errores en Bash:** Los tres programas en Bash gestionan adecuadamente los argumentos posicionales, rutas con espacios, ramas condicionales en fronteras (≤ 20 vs > 20), y terminan con código de salida no cero ante ficheros ausentes o variables vacías.
- **1,5 puntos · Transformación dirigida con sed:** Aplicación precisa de expresiones regulares y direcciones en `sed` para transformar únicamente las cabeceras FASTA sin alterar las líneas de secuencia ni sobrescribir archivos originales.
- **1,5 puntos · Procesamiento estructurado y agregación con awk:** Exclusión rigurosa de cabeceras, suma correcta de longitudes de secuencia y justificación del resultado global mediante contraste manual sobre un registro testigo.

- **1,5 puntos · Defensa razonada y justificación metodológica:** Explicación clara (150–200 palabras) sobre la necesidad del fallo explícito frente a la finalización silenciosa, demostrando comprensión del impacto operativo en flujos bioinformáticos automatizados.

Nota didáctica: La verificación sintáctica con `bash -n` y la obtención de `DELIVERY_OK` son requisitos de entrada indispensables, pero no sustituyen el diseño correcto de ramas lógicas ni la defensa metodológica.

Lista de autocorrección

- distingo *glob* de regex y declaro `-E` cuando uso ERE;
- no analizo `ls` para contar ficheros;
- cada variable que representa una ruta está entrecomillada;
- compruebo cero entradas y fichero ausente antes de confiar en la salida;
- no escribo resultados encima de las entradas;
- `bash -n` pasa en los cinco scripts;
- declararé siempre a qué líneas se aplica cada sustitución de `sed`;
- probé las ramas ≤ 20 , > 20 y fichero ausente;
- contrasté al menos un conteo con un caso mínimo calculado a mano;
- `check_practice_delivery.sh` termina con `DELIVERY_OK`.

Defensa

En 150–200 palabras, explique por qué el programa 3 debe fallar de forma explícita ante un fichero inexistente en vez de no imprimir nada, y qué diferencia práctica tiene eso para quien ejecute su script sin leer el código.

Fuentes

El corpus está en `data/corpus/` (`sources_manifest.csv`, TC02-C5-DATA, CC0-1.0), reutilizado de la práctica histórica del módulo legacy LEG-TC01 bajo procedencia propia.

Bibliography

- [1] Vince Buffalo. *Bioinformatics Data Skills: Reproducible and Robust Research with Open Source Tools*. O'Reilly Media, 2015. Pasajes comprobados en las páginas físicas 74–79 y 170–180 del PDF local.
- [2] GNU Project. *GNU Awk User's Guide*, 2026. Consultado el 7 de agosto de 2026. https://www.gnu.org/software/gawk/manual/html_node/index.html.
- [3] GNU Project. *GNU sed User's Guide*, 2026. Sintaxis de s///, direcciones y -i. Consultado el 7 de agosto de 2026. <https://www.gnu.org/software/sed/manual/sed.html>.
- [4] National Center for Biotechnology Information. *FASTA Format for Nucleotide Sequences*, 2026. Documentación oficial. Consultada el 7 de agosto de 2026. <https://www.ncbi.nlm.nih.gov/genbank/fastafORMAT>.